SynodSQL: A Git-Style Replication Model for Fully Replicated, Trusted Peer Databases

Oct 12, 2026 · @Jean-Luc Robitaille

Abstract

This paper describes SynodSQL, a replication model for a small network of trusted servers. Every server holds a full copy of all data and writes locally at all times. At fixed intervals (epochs), all servers merge their changes into one shared state.

The merge is an N-way git-style merge with one common base: the state at the end of the previous epoch. The merge function is deterministic, so every server computes the same result without a central merge step. A Merkle root hash of the result lets the servers prove that they agree.

The paper gives the storage layout, the sync protocol, the merge algorithm, a taxonomy of write conflicts with a rule for each type, a worked example, and a model of the conflict rate. Conflicts that rules cannot resolve become rows in a replicated conflict table. They never stop the merge. The model gives up global serializability. In exchange, it gives local writes with no network wait, full offline operation, and simple recovery.

1. Motivation

Early ARPANET design had one clear goal: resource sharing among a set of known, trusted hosts [6]. The hosts belonged to research institutions that trusted each other. Shared data, such as the host table HOSTS.TXT, was copied to every site [3].

HOSTS.TXT had one writer. Every other host only read it. This design failed as the network grew, because the single writer became a bottleneck. DNS then split ownership into zones.

This paper asks a different question: what if every trusted host can write, and every host keeps all data? Each host then works at local speed, and it continues to work when the network is down. The cost is that hosts must merge their changes, and concurrent changes can conflict.

Version control systems solved a similar problem for source code. Git lets many people change copies of the same files and merge them later. This paper applies the git model to database rows, with three changes:

1. The merge runs automatically on a fixed schedule, not by hand.
2. Every host computes the merge itself, and all hosts get the same result.
3. A conflict does not block the merge. It becomes data that people or programs resolve later.

2. Assumptions and scope

The model assumes a small, closed group of trusted nodes that hold all data. Section 13 discusses what fails when an assumption is false.

Assumption	Meaning	Consequence
Trusted nodes	No node lies or attacks. Nodes can still crash, have bugs, or lose disks.	No signatures or Byzantine fault tolerance. Checksums are still necessary.
Closed membership	A known set of N nodes. Peers authenticate with a shared secret or mutual TLS.	Outsiders cannot join or inject data.
Full replication	Every node stores all data.	Storage per node equals the full database size.
Small N	Tens of nodes, up to low hundreds.	All-to-all exchange per epoch is acceptable.
Periodic sync	The merge runs once per epoch (minutes to one day).	Other nodes see a write after at most one epoch plus sync time.
Local-first writes	A write commits locally with no network wait.	No global serializability.
Loosely synced clocks	Clocks drift. Hybrid logical clocks (HLC) [4] limit the effect.	Tie-breaks are deterministic and close to real-time order.

Out of scope: open networks, untrusted peers, partial replication, and workloads that need strict global invariants at every instant (Section 7.6).

3. Storage model

Each node stores the database in one file, similar to SQLite. Only row content is shared between nodes. Everything else in the file is local and derived, so each node can rebuild it at any time.

3.1 Heap pages

Table rows live in unordered heap pages. Each row has a header and column values:

```
row_id     128-bit logical ID = origin node ID (32 bits) + local counter (64
bits) + reserved (32 bits)
stamp[c]   per-column HLC timestamp + node ID of the last write to column c
flags      live | tombstone
values[c]  column values
```

The row ID never changes. Two nodes cannot make the same ID, so a merge never renumbers rows. The per-column stamps let the merge compare a late write with the newest write to that column (Section 11.1).

3.2 Row map

A B-tree maps row_id to (page, slot). It is local, because physical addresses are different on each node.

3.3 Secondary indexes

B-trees map column values to row_id. They are local and derived. After each merge, a node updates them for the changed rows, or bulk-loads them again. A rebuild is also a chance to apply an access-weighted layout that puts frequently read keys near the root.

3.4 State hash tree

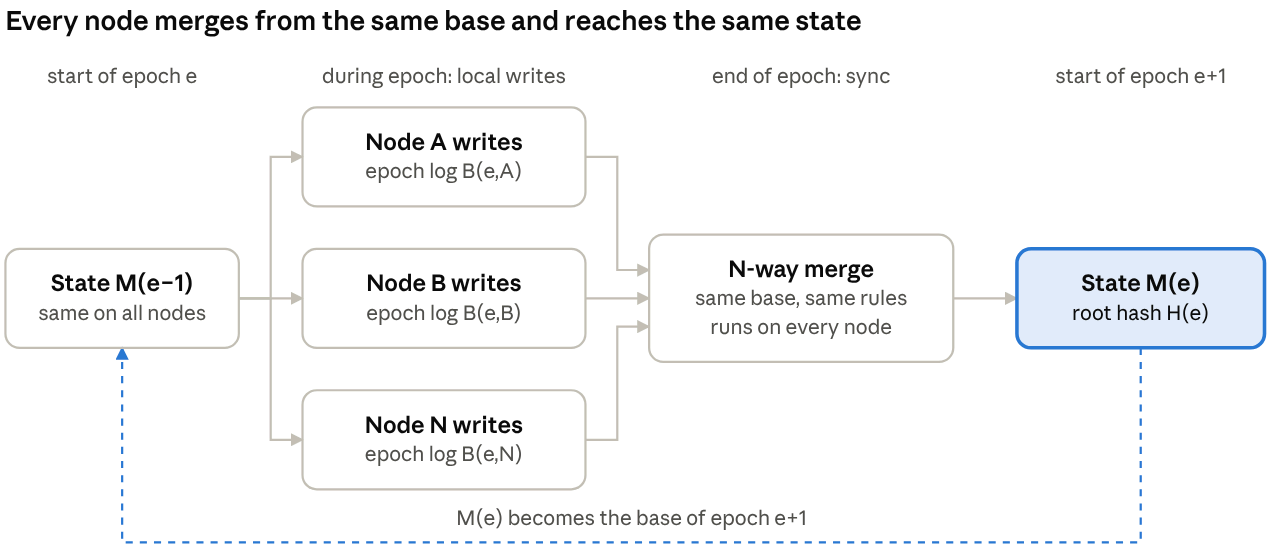
A Prolly tree (a content-defined Merkle B-tree, as in Dolt [11]) maps row_id to hash(row content). Its node boundaries come from content hashes, so the same data always gives the same tree, whatever the order of inserts. The root hash therefore identifies the full shared state. Two nodes with the same root hash hold the same data. Two nodes with different hashes can find the differing rows by a walk down the differing subtrees only.

3.5 Epoch log

During an epoch, every local write also goes to an epoch log. Each entry holds the row ID, the operation, the transaction ID, the HLC time, the before-image, and the after-image. The log is the node's branch for this epoch. Its size is proportional to the rows that the node changed, not to the database size. The before-images also keep the epoch base readable, so the node does not need a second copy of the database.

4. The epoch model

Time is divided into epochs; every epoch starts from the same shared state on every node and ends with one N-way merge.



epoch flow · N logs, one base, one merge

Each node writes to its own epoch log. At the end of the epoch, every node runs the same merge on the same inputs and gets the same root hash.

Terms:

- **M(e−1):** the merged state at the start of epoch e. All nodes hold an identical copy.
- **B(e,i):** the epoch log of node i in epoch e, which holds its changes against M(e−1).
- **M(e) = merge(M(e−1); B(e,1), …, B(e,N)):** the merged state at the end of epoch e.

The key simplification: every log in epoch e has the same merge base, M(e−1). Git must search a commit graph to find a merge base. Here the base is always known, so the merge is a simple three-way comparison for each row.

4.1 The git analogy

Git	SynodSQL
Repository	Database file on one node
Commit	Local transaction
Branch	A node's epoch log B(e,i)
Merge base	Previous merged state M(e−1)
Octopus merge	The N-way epoch merge
Merge driver (.gitattributes)	Column merge policy in the schema
Conflict markers in a file	Rows in the _conflicts table
Rebase	Late and in-flight writes applied on top of the new M(e)
Revert	A revert log in a later epoch
Clone	Full copy for a node more than L epochs behind
Tree hash	Prolly tree root hash

Two differences matter. First, in git one person makes a merge commit and pushes it. Here every node computes the merge, and the root hash proves that all nodes agree. Second, a git conflict stops the merge until a person fixes it. Here a conflict never stops the merge.

5. Sync protocol

Each epoch has a write phase and a sync phase. The sync phase has five steps, and only step 3 needs any coordination.

1. **Seal.** At the epoch boundary (by HLC time), each node closes its epoch log B(e,i). New local writes go to the next log, B(e+1,i). The node computes a hash of the sealed log.
2. **Exchange.** Each node sends its sealed log to the other nodes. For small N, every node sends to every other node. For larger N, nodes use gossip: each node forwards the logs it has to a few random peers in each round. Gossip reaches all nodes in about $\log_2 N$ rounds [2].

3. **Agree on the manifest.** All nodes must merge the same set of logs, or they compute different states. The epoch leader (node e mod N, or the next node in order if that node is down) publishes a manifest: the list of log hashes in epoch e. A log that does not arrive before the deadline goes into a later epoch (Section 11.1).
4. **Merge.** Each node runs the deterministic merge (Section 6) on M(e−1) and the logs in the manifest. The result is M(e), the conflict rows for epoch e, and the root hash H(e).
5. **Verify and switch.** Nodes exchange H(e). If the hashes match, each node makes M(e) its new base, re-applies its in-flight writes from B(e+1,i) on top of M(e), and updates its local indexes. A node with a different hash has a bug or bad data. It fetches the differing subtrees of M(e) from a peer and raises an alert.

In-flight writes. Writes made during the sync phase are in B(e+1,i). Each entry keeps its own before-image and HLC time. The next merge treats them by the same rules as late writes (Section 11.1), so no special rebase logic is necessary.

Split leaders. During a network split, two nodes can each act as leader and publish different manifests. Nodes accept the manifest from the highest-priority leader they can reach. If a node later sees a higher-priority manifest for the same epoch, it discards its M(e) and merges again. This is safe, because M(e) is derived data and no log is lost. For strict agreement, the manifest can go through a consensus log such as Raft [5]. A manifest is only a list of hashes, so the cost is low.

Traffic. With all-to-all exchange, total traffic per epoch is about $N \times (N - 1) \times \Delta$, where Δ is the mean log size. With gossip or relays, each node sends about $\Delta \times$ fanout. Every node already holds M(e−1), so no node sends the full database in normal operation.

6. The merge algorithm

The merge is a pure function of M(e−1) and the logs in the manifest. It does not depend on which node runs it, on arrival order, or on local clocks. This gives the main property of the model: all nodes that use the same manifest get the same M(e).

6.1 Per-row merge

For each row that at least one log changed:

1. Collect the changes from all logs and sort them by node ID. If one node changed the row more than once, keep only its last after-image, with its first before-image.
2. If only one node changed the row, take its after-image.
3. If more than one node changed the row:

- If any change is a delete, apply the delete rule (Section 7.3).
- Else, merge column by column. A node changed a column if its after-image differs from its before-image.
- No node changed the column: keep the current value.
- All changing nodes wrote the same value: take that value.
- Nodes wrote different values: apply the column policy (Section 7.2).

Row IDs are unique, so two nodes never insert the same row. An insert always enters step 2.

6.2 Constraint pass

After all rows merge, the merge checks unique keys, foreign keys, and check constraints on M(e). Each violation becomes a constraint conflict (Sections 7.4 to 7.6). The fix for each violation must also be deterministic.

6.3 Pseudocode

```
merge(base, logs):                          # logs sorted by node_id
  changes = group all log entries by row_id
  result = copy_on_write(base)
  conflicts = []
  for row_id in sorted(changes):
    cs = changes[row_id]                    # entries: node, op, before, after,
hlc, txn
    if len(cs) == 1:
      apply(result, cs[0])
    elif any(c.op == DELETE for c in cs):
      resolve_delete(result, row_id, cs, conflicts)
    else:
      row = copy(result[row_id])
      for col in columns(row_id):
        writes = [c for c in cs if c.after[col] != c.before[col]]
        if not writes:
          continue
        if all_equal(c.after[col] for c in writes):
          row[col] = writes[0].after[col]
        else:
          row[col] = policy(col).resolve(row[col], writes, conflicts)
```

```
        result.put(row_id, row)
    check_constraints(result, conflicts)    # unique, foreign key, check
    conflicts.sort(key=(table, row_id, column))
    return result, conflicts, root_hash(result)
```

6.4 Determinism rules

- Each policy must give the same result for the same set of inputs, in any order. Last-writer-wins with a total order on (HLC, node ID), max, min, sum of deltas, and set union all qualify.
- A policy that depends on order (for example, a text merge) runs in node ID order.
- No policy may read the local clock, local indexes, random numbers, or any data outside M(e−1) and the logs.
- Sums of floating-point values must use a fixed order, because float addition is not associative. Use decimal or integer types for money and quantities.

Cost. Sorting the changes costs $O(\Delta \log \Delta)$, where Δ is the total log size. Index updates and constraint checks cost $O(\Delta \log n)$. The merge cost depends on how much changed, not on the database size n.

7. Conflict taxonomy and resolution

Most concurrent writes merge automatically; only unique keys, foreign keys, and cross-row invariants need a person. The table lists every conflict type the model can produce, from common and easy to rare and hard.

Section	Type	Example	Default rule	Needs a person?
7.1	Disjoint rows	A changes row 1, B changes row 2	Take both	No
7.1	Same row, different columns	A changes price, B changes name	Take both columns	No
7.2	Same column, different values	A sets price 0.12, B sets 0.11	Column policy	Only with a flag policy

Section	Type	Example	Default rule	Needs a person?
7.3	Update and delete	A deletes a row, B updates it	Delete wins, B's version recorded	Optional
7.4	Unique key collision	A and B each insert a part with SKU C-400	Earliest insert keeps the key	Yes
7.5	Orphan foreign key	A inserts an order for customer 7, B deletes customer 7	Undo the delete, record it	Yes
7.6	Broken invariant	Stock is 5, A sells 4, B sells 3	Keep result, record it, or prevent with escrow	Yes, or design it out
7.7	Schema change	A adds a column, B renames it	Not allowed in normal epochs	Not applicable

7.1 No-conflict cases

Most concurrent writes touch different rows or different columns. A column-level merge removes a large class of false conflicts that a row-level merge would report. Git works at line level for the same reason: two edits to different lines of one file merge cleanly.

7.2 Same column, different values: column policies

Each column declares a merge policy in the schema, like a git merge driver in .gitattributes:

```
CREATE TABLE inventory (
  row_id  ROWID,
  sku     TEXT UNIQUE                MERGE lww,
  name    TEXT                       MERGE lww,
  price   DECIMAL(10,2)              MERGE lww_flag,
  qty     INTEGER CHECK (qty >= 0)   MERGE delta,
  tags    SET<TEXT>                  MERGE union,
  notes   TEXT                       MERGE text3
);
```

Policy	Rule	Good for	Risk
lww	Highest (HLC, node ID) wins	Names, labels, status	The other write is lost silently
lww_flag	Same as lww, and the losing values go to _conflicts	Prices, settings that matter	Review work
delta	current + Σ (after − before) over all writers	Counters, quantities, balances	Can break a check constraint (7.6)
max, min	Largest or smallest value	High-water marks, last-seen times	Concurrent edits cannot lower (or raise) the value
union	(current ∪ all additions) − all removals	Tags, member lists	A removal on one node wins over the item staying on another
text3	Line-level three-way text merge, in node ID order	Notes, long text	Overlapping edits become conflicts
flag	Keep the current value, record all values	Data that must never change silently	Most review work

The delta policy matters most. With lww, a sale on node A and a sale on node B produce a lost update: one sale disappears. With delta, both sales count. This is the same idea as a PN-counter CRDT [7]. The common base also makes union clean: within one epoch, a node can only remove items that were in the base, and it can only add items that were not, so an add and a remove of the same item cannot both occur.

7.3 Update and delete

Node A deletes a row. Node B updates the same row in the same epoch. Each table selects one of three rules:

- **Delete wins (default).** The row becomes a tombstone. B′s after-image goes to _conflicts.
- **Update wins.** The row stays, with B′s values. A′s delete goes to _conflicts.
- **Flag.** The row stays as a tombstone with B′s values attached, and a person decides.

Delete wins is the default because a delete is usually deliberate. The record in _conflicts means that no data is lost.

7.4 Unique key collision

Row IDs never collide, but user values can. Node A and node B each insert a part with SKU C-400. The merged table now has two rows with one unique value, and the merge cannot know which row is right.

Rule: the row whose insert has the earliest (HLC, node ID) keeps the value. The other row moves to _conflicts, with a reference to the winner. A person or program then merges the two rows, renames one, or discards one.

Prevention: use generated keys where possible. Where users must choose keys, give each node a key range or prefix. For example, only node A makes SKUs that start with A-.

7.5 Orphan foreign key

Node A inserts an order for customer 7. Node B deletes customer 7. Each change is valid on its own node, but together they break the foreign key.

Rule: undo the delete of the parent row (restore it from $M(e-1)$), and record a conflict that names both changes. This loses no data. A table can select cascade delete instead.

7.6 Invariants across rows or values

Some rules cannot survive a merge. Stock is 5. Node A sells 4, and node B sells 3. Each sale is valid locally, because each node saw 5 in stock. The delta merge gives $5 - 4 - 3 = -2$, which breaks $qty \geqslant 0$.

No merge rule can fix this after the fact, because the goods are already sold twice. Bailis et al. call the needed property invariant confluence (I-confluence): an invariant is safe under merge only if a merge of two valid states always gives a valid state [1].

Invariant	Safe under merge?
Unique generated ID	Yes
Unique user-chosen value	No
Foreign key, inserts only	Yes
Foreign key, with deletes	No

Invariant	Safe under merge?
qty ⩾ 0, increments only	Yes
qty ⩾ 0, with decrements	No
start < end, columns changed on different nodes	No

For an unsafe invariant, a table selects one of three approaches:

1. **Accept and repair.** Keep the merged result and record a conflict. A person or program repairs it later, for example with a backorder or a refund. Many real businesses already work this way.
2. **Escrow.** Split the resource between nodes before the epoch. With stock 5 and two nodes, A may sell up to 3 and B up to 2. Each node checks its own share locally. The merge then gives out new shares by a fixed rule, for example in equal parts. The invariant holds with no coordination during the epoch.
3. **Coordinate.** Send that one operation to a single owner node or a consensus group. This gives up local writes for that operation only.

7.7 Schema changes

Two nodes that change the schema in different ways produce data that cannot merge. So schema changes do not go through normal epochs. A schema change is a special manifest entry. All nodes must acknowledge it before the epoch that applies it. In that epoch, the merge first migrates M(e−1) and every log entry with the same migration function, and then merges as usual.

7.8 The _conflicts table

Each recorded conflict becomes one row:

```
_conflicts(
  conflict_id    hash of (epoch, table, row_id, column, kind), the same on
all nodes
  epoch, table, row_id, column, kind,
  base_value,
  values         list of (node_id, hlc, value)
  applied_value, rule,
  status         open | resolved
)
```

The _conflicts table is a normal replicated table. Any node can resolve a conflict: it writes the chosen value to the data row and sets status to resolved. The resolution travels in the next epoch like any other write. If two nodes resolve the same conflict differently in one epoch, the result is a new same-column conflict, which lww_flag handles. In practice this ends quickly, because it needs two people to act on the same conflict in the same epoch.

8. Worked example

In one epoch, three nodes make nine writes to an inventory table; the merge completes with no stop and records three conflicts for review. The table uses the policies from Section 7.2 (name lww, price lww_flag, qty delta, sku unique) and the delete-wins rule.

Base state M(e−1):

row_id	sku	name	price (USD)	qty
r1	A-100	Bolt, M6	0.10	500
r2	A-200	Nut, M6	0.05	800
r3	B-310	Washer, M6	0.02	1000

Writes during epoch e:

Node	HLC	Row	Change
A	10	r1	price 0.10 → 0.12
B	11	r1	name → Bolt, M6 x 20
A	12	r2	qty 800 → 650 (sold 150)
B	13	r2	qty 800 → 700 (sold 100)
A	14	r3	delete
B	15	r3	qty 1000 → 900
B	16	r4 (new)	insert C-400, Spacer, 0.08, qty 200
C	17	r1	price 0.10 → 0.11
C	18	r5 (new)	insert C-400, Spacer 10 mm, 0.09, qty 50

Merge, row by row:

- **r1.** Only B changed name, so the merge takes it. A and C changed price to different values. Policy lww_flag: C has the later HLC (17 against 10), so price becomes 0.11. A's value 0.12 goes to _conflicts.
- **r2.** A and B both changed qty. Policy delta: 800 + (650 − 800) + (700 − 800) = 550. Both sales count, with no conflict. With lww, the result would be 700, and A's sale of 150 would disappear.
- **r3.** A deleted the row, and B updated it. Delete wins. B's after-image (qty 900) goes to _conflicts.
- **r4, r5.** The row IDs are different, so the per-row pass takes both. The constraint pass then finds two rows with SKU C-400. B's insert has the earlier HLC (16 against 18), so r4 keeps the key. r5 moves to _conflicts.

Result M(e):

row_id	sku	name	price (USD)	qty
r1	A-100	Bolt, M6 x 20	0.11	500
r2	A-200	Nut, M6	0.05	550
r4	C-400	Spacer	0.08	200

_conflicts for epoch e:

Target	Kind	Applied	Recorded	Rule
r1.price	Same column	0.11 (C)	0.12 (A)	lww_flag
r3	Update and delete	Deleted (A)	qty 900 (B)	Delete wins
r5	Unique key C-400	r4 keeps C-400 (B)	r5, Spacer 10 mm (C)	Earliest insert

All three nodes compute these exact tables and the same root hash H(e). A person now reviews three conflicts. The other changes, including both concurrent sales on r2, needed no review.

9. Conflict rate analysis

Conflicts per unit time grow linearly with epoch length and with the square of the node count, so shorter epochs and small groups keep review work low.

Model. N nodes. In one epoch, each node writes w rows. Each write goes to row i with probability p_i. A row conflicts if two or more nodes write it in the same epoch. When w × p_i is small, the chance that one node writes row i is about w × p_i, and the expected number of conflicting rows per epoch is:

```
E[\text{conflicting rows}] \approx \binom{N}{2}\, w^2 \sum_i p_i^2
```

For uniform access over R rows, the sum of p_i^2 is 1/R. With a write rate of r per node and an epoch length of T, w = rT. Conflicts per unit time are then:

```
\frac{E[\text{conflicting rows}]}{T} \approx \frac{N(N-1)\, r^2\, T}{2R}
```

Results.

1. The conflict rate grows linearly with epoch length T. Halving T halves the conflicts per day.
2. It grows with N^2, so the model suits small groups.
3. Skew dominates. The sum of p_i^2 is the collision probability of the access pattern. If 1% of all writes go to one row, that one row alone adds 10^{-4}, the same as uniform access over only 10,000 rows.

Example. N = 5 nodes, r = 1,000 writes per hour per node, R = 1,000,000 rows, uniform access.

Epoch length	Writes per node per epoch (w)	Conflicting rows per epoch	Conflicting rows per day
1 day	24,000	about 5,760	about 5,760
1 hour	1,000	about 10	about 240
5 minutes	83	about 0.07	about 20

These are row-level conflicts. Column-level merge and mergeable policies (delta, max, union) resolve most of them with no record. Only flag-type policies and constraint conflicts reach a person.

Design rules.

- Make hot columns mergeable: use delta, max, or union on columns that many nodes write.
- Choose the epoch length from the acceptable conflict rate, not only from bandwidth.
- Keep user-chosen unique keys out of hot paths, because those conflicts always need review.

10. Correctness and consistency properties

The model guarantees convergence and bounded staleness; it does not guarantee global serializability.

Guaranteed:

- **Convergence.** Two nodes that apply the same sequence of manifests reach the same state. Proof sketch: M(0) is the same on all nodes. Each M(e) is a deterministic function of M(e−1) and the logs in manifest e. The manifest names logs by hash, so all nodes use the same inputs. By induction, all M(e) are equal. The root hash check in step 5 detects any break in this chain, such as a bug or a bad disk.
- **Local durability.** A write is durable on its node when the local transaction commits.
- **Bounded staleness.** A write in manifest e is visible on all live nodes after the sync of epoch e. The bound is one epoch plus sync time.
- **Read your writes.** A node always sees its own writes, also before the merge.
- **Local isolation.** Transactions on one node are serializable with respect to that node's view, as in SQLite.
- **No lost updates on mergeable columns.** The delta, max, min, and union policies keep the effect of every write.

Not guaranteed:

- **Global serializability.** Two nodes can each run a transaction that is valid locally, and the merged result may not equal any serial order. Write skew is possible, and so are lost updates on lww columns.
- **Invariants that are not I-confluent** (Section 7.6). The model detects and records each break, but it does not prevent it.
- **Atomic transactions across the merge.** Conflict rules apply per row. A transaction that changed rows r1 and r2 can win on r1 and lose on r2. A group of tables can opt in to transaction-level resolution: if any row of a transaction loses, all rows of that

transaction go to _conflicts. The transaction ID in each log entry makes this possible, and it stays deterministic.

In CAP terms, the model chooses availability. Every node accepts reads and writes during a network partition. Consistency is eventual, with a bound of one epoch while the network is up.

11. Failures, late nodes, and garbage collection

Missing logs wait for a later epoch, nodes that fall too far behind clone a peer, and tombstones stay until no older log can arrive.

11.1 A node misses the deadline

Node k is offline at the end of epoch e, so its log B(e,k) is not in manifest e. The other nodes merge without it. When node k comes back, it fetches the current state from a peer. The Prolly tree diff sends only the subtrees that changed. Node k then submits B(e,k), which goes into the next manifest as a late log.

The merge handles a late log with the normal rules, because each log entry carries its own before-image and HLC time:

- **delta** adds the entry's own change (after − before) to the current value. The late sale still counts.
- **lww** compares the entry's HLC with the column stamp in the current state. An old write loses to any newer write, which is the correct result.
- A column counts as changed if the entry's after-image differs from its own before-image, not from the current value.

In-flight writes from the sync phase (Section 5) use the same mechanism.

Lateness bound L. A node that is more than L epochs behind cannot submit its old logs as normal writes, because the tombstones they depend on may be gone (11.3). It clones the current state of a peer. Its old log entries go to _conflicts as stale writes for review.

11.2 Crash during the merge

The merge writes M(e) beside M(e−1) by copy-on-write. The switch is one atomic pointer update in the file header. After a crash before the switch, the node still has M(e−1) and the logs, so it runs the merge again. The result is the same, because the merge is deterministic.

11.3 Tombstones and garbage collection

A deleted row stays as a tombstone, so that a late write cannot bring it back. The Xerox Clearinghouse used the same idea and called the records death certificates [2]. A tombstone from epoch e can be removed after epoch e + L, because no log older than L epochs is accepted. The same bound lets nodes drop old logs, old manifests, and resolved _conflicts rows.

11.4 Leader failure

If the epoch leader is down, the next node in order publishes the manifest after a timeout. Section 5 describes the rule for two leaders during a network split.

11.5 A node with a bug

A bug in the merge on one node gives a different root hash, and step 5 detects it. A bug that writes bad data into a log is harder, because the log is valid input. All logs and manifests stay available for L epochs, so the nodes can undo the damage with a revert log in a later epoch, like git revert.

12. Related work

Most parts of this model exist in earlier systems. The contribution is the combination: one common merge base per epoch, a deterministic merge on every node, and conflicts as replicated rows that never block the merge.

System	What it shares with this model	Main difference
Xerox Clearinghouse, epidemic algorithms [2]	Fully replicated database on trusted servers; anti-entropy; death certificates	Last-writer-wins only; no conflict records
Bayou [8]	Weakly connected replicas; merge procedures per application; dependency checks	A primary server fixes the final order; writes stay tentative until then
Lotus Notes	Replication between trusted servers; conflict documents kept for review	Document level, not column level

System	What it shares with this model	Main difference
CouchDB	Every replica picks the same winning revision; losing revisions kept as conflicts	Document level; no common base per epoch
Git [10]	Branches, three-way merge, merge drivers, rebase, revert	Merges by hand; a conflict blocks the merge
Dolt [11]	SQL with git-style branches, cell-level three-way merge, conflict tables, Prolly trees	Merges are explicit user actions, not automatic merges among peers
cr-sqlite and CRDTs [7, 12]	Merge without coordination; per-column last-writer-wins and counters	No conflicts are recorded; every merge is silent
Calvin [9]	Replicas stay identical because they run deterministic logic on agreed input	Agrees on input order before execution, so writes wait for the network
Invariant confluence [1]	Theory of which invariants survive a merge	Theory, not a system

The closest single system is Dolt. SynodSQL is roughly Dolt with an automatic, scheduled octopus merge among peers, where every node computes the merge and the root hash proves agreement.

13. Limitations and open questions

The model fits a small group of trusted nodes with mostly mergeable data, and it fits badly where those assumptions fail.

Limitations:

- **Storage.** Every node holds the full database, so total storage is N times the data size.
- **Scale.** Traffic and conflicts grow with N^2. Above a few hundred nodes, use relays or partition the data.

- **Contended invariants.** Bank balances, seat bookings, and stock with hard limits need escrow or coordination (Section 7.6). The model detects the problem but does not remove it.
- **Staleness.** Other nodes see a write only after the merge. Some workflows cannot accept this.
- **Review load.** lww_flag, flag, unique-key, and foreign-key conflicts need people. Section 9 shows that a busy system with long epochs can make thousands per day.
- **Trust.** One node with a bug can write bad data to all nodes. A revert repairs the data, but a person must first see the problem.
- **Clock dependence.** The lww order follows HLC time. Within the HLC drift bound, a node with a fast clock wins more contests.

Open questions:

1. What epoch length is best for a given write rate, bandwidth, and review budget? Section 9 gives a starting point.
2. Can the leader and manifest step go away? One option: include every log sealed before an HLC deadline, and merge again when a late log arrives.
3. Can transaction-level conflict resolution stay cheap for large transactions?
4. How well do automatic escrow rules work on real inventory data?
5. Does an access-weighted index layout, rebuilt at each merge, give a measurable read gain over a plain B-tree?

14. Prototype and evaluation plan

Nothing in this paper is measured yet; a prototype on SQLite can test every main claim. The merge layer can be a Python or Rust library on top of SQLite.

Prototype parts:

1. Logical row IDs and per-column HLC stamps, as extra columns on each SQLite table.
2. An epoch log, filled by triggers that copy the before-image and after-image of every write.
3. A Prolly tree over (row_id, row hash) for the root hash and for catch-up diffs.
4. The merge function, with the policies in Section 7.2 and a _conflicts table.
5. Sync over HTTP with mutual TLS, and a fixed leader rotation for manifests.

Experiments:

Claim	Experiment	Metric
Convergence	5 to 50 nodes, random writes, random crashes and late nodes, 10,000 epochs	Root hash mismatches (target: 0)
Determinism	Property-based test: random concurrent histories, random arrival order on each node	Identical M(e) on every node
Conflict model (Section 9)	Uniform and Zipf workloads; vary N, T, and skew	Measured against predicted conflicting rows
Merge cost	Vary total log size Δ and database size n	Merge time; check that it follows Δ, not n
Sync cost	Vary N, fanout, and Δ	Bytes sent per node per epoch
Value of column policies	Same workload with lww only, then with all policies	Lost updates; conflicts that need review
Index rebuild	Plain B-tree against an access-weighted layout, rebuilt at each merge	Read latency at p50 and p99

15. Conclusion

A small network of trusted servers can share one database with full copies, local writes, and no central server. The key idea is a global epoch: all nodes merge from the same base with the same deterministic rules, so they reach the same state without a merge coordinator. Column policies resolve most concurrent writes automatically. The rest become rows in a replicated conflict table, and they never block work. The model gives up global serializability and invariants that are not I-confluent. In exchange, it gives the ARPANET goal of shared resources among trusted peers, with the merge discipline of git.

References

These references are from memory and were not checked against the original sources.

1. Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., Stoica, I. Coordination Avoidance in Database Systems. PVLDB 8(3), 2014.

2. Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., Sturgis, H., Swinehart, D., Terry, D. Epidemic Algorithms for Replicated Database Maintenance. PODC, 1987.
3. Harrenstien, K., Stahl, M., Feinler, E. DoD Internet Host Table Specification. RFC 952, 1985.
4. Kulkarni, S., Demirbas, M., Madappa, D., Avva, B., Leone, M. Logical Physical Clocks. OPODIS, 2014.
5. Ongaro, D., Ousterhout, J. In Search of an Understandable Consensus Algorithm. USENIX ATC, 2014.
6. Roberts, L. G., Wessler, B. D. Computer Network Development to Achieve Resource Sharing. AFIPS Spring Joint Computer Conference, 1970.
7. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. Conflict-Free Replicated Data Types. SSS, 2011.
8. Terry, D. B., Theimer, M. M., Petersen, K., Demers, A. J., Spreitzer, M. J., Hauser, C. H. Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System. SOSP, 1995.
9. Thomson, A., Diamond, T., Weng, S.-C., Ren, K., Shao, P., Abadi, D. J. Calvin: Fast Distributed Transactions for Partitioned Database Systems. SIGMOD, 2012.
10. Chacon, S., Straub, B. Pro Git, 2nd edition. Apress, 2014.
11. Dolt: a version-controlled SQL database (DoltHub). Project documentation.
12. cr-sqlite: a CRDT extension for SQLite (vlcn.io). Project documentation.